

Functional Prototypes - UI Software

Diogo Cabral
diogo.cabral@isel.pt

Functional Prototypes

A **functional prototype** captures both function and appearance of the intended design, though it may be created with different techniques and even a different scale from the final design.

Functional Prototypes

How to implement functional prototypes?

Figma?

Most of design tools are limited to GUI

Do not cover multimedia/multimodal interactions

Coding tools

Easy Frameworks (facilitate event handling):

Processing (Java)

openFrameworks (C++)

-> but miss GUI Toolkits

Solution

Use regular dev. tools for functional prototyping.

(also allows log files, useful for evaluation -> TBD)

UI Software

How to integrate a UI into an application?

First:

Software architecture patterns that integrate UIs.

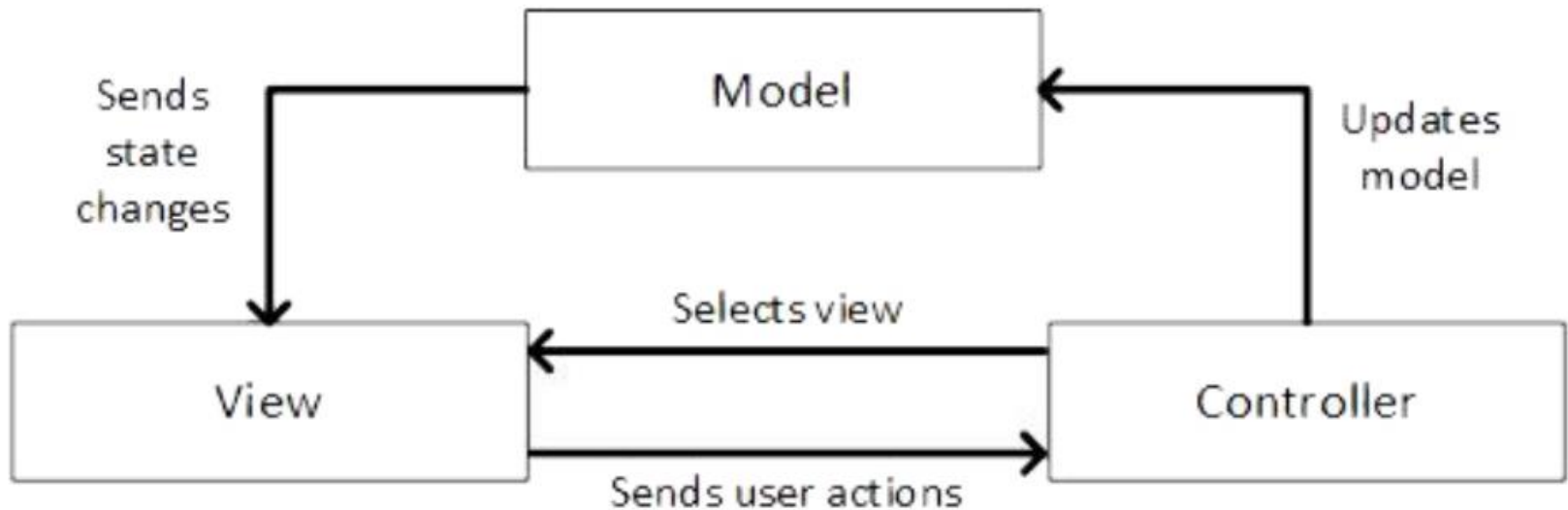
A software architecture pattern is a solution to a well-understood recurring problem in a particular context. It provides a high-level structure and behavior for software systems.

It is a grouping of design decisions that have been repeated and successfully used in a given context.

Software architecture patterns provide the structure and main components of the software system being built.

UI Software

Model-View-Controller (MVC) Pattern



UI Software - MVC

Model

The model manages the application data and the state. Among its responsibilities is the processing of data to and from a data store, such as a database. Models receive directives from controllers to retrieve and update data. In some variations of MVC, the model is passive and must receive a request to send out an application state update. In other variations, a view may be active and push notifications of model state changes to a view.

UI Software - MVC

View

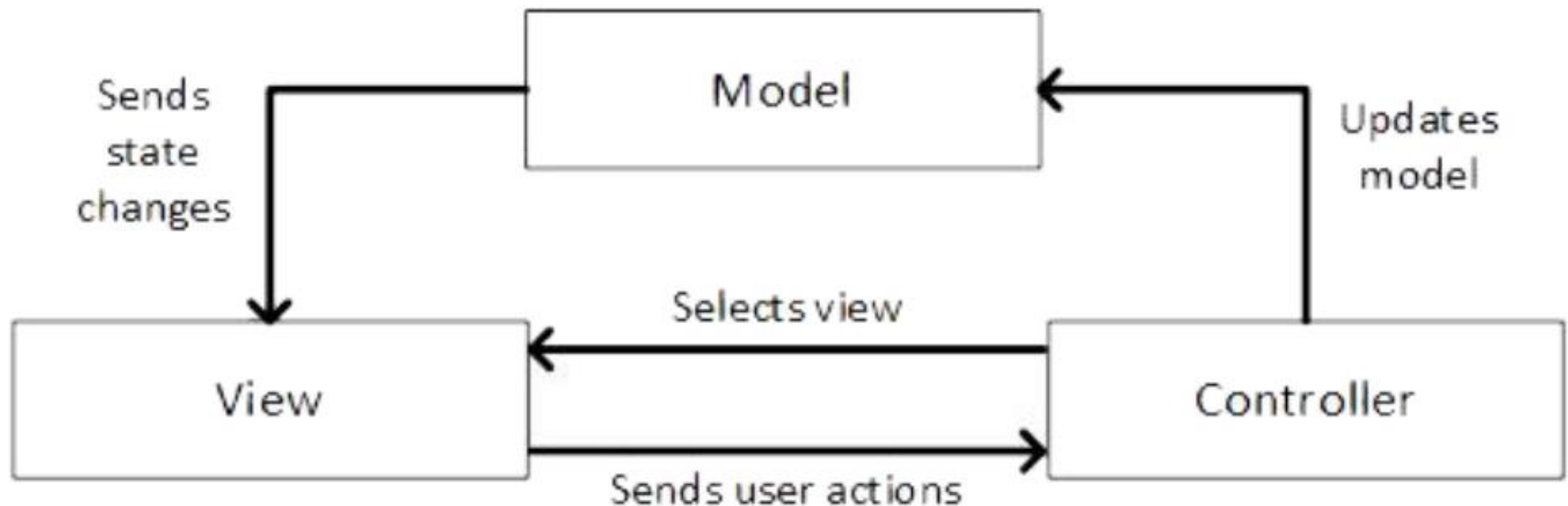
The view is responsible for the presentation of the application. It is the part of the application that is visible to the user. The view displays data to the user in an appropriate interface based on information received from the controller. If the model is providing application state updates directly to views, the views may also be updated based on these notifications. As users manipulate a view, such as providing input or providing some user action, the view will send this information to a controller.

UI Software - MVC

Controller

A controller acts as an intermediary between the model and the view. A controller executes application logic to select the appropriate view and sends it the information that it needs to render the user interface. Views notify controllers of user actions so that the controller can respond to them. Controllers will update the model based on user actions.

UI Software - MVC



MVC – C# Example

// Model — pure data

```
class PlayerModel {  
    public int Health { get; set; } = 100;  
}
```

// View — display only

```
class PlayerView {  
    public void Render(int health) => Console.WriteLine($"Health: {health}");  
}
```

// Controller — handles input, updates model, tells view to render

```
class PlayerController {  
    PlayerModel model = new PlayerModel();  
    PlayerView view = new PlayerView();  
  
    public void TakeDamage(int amount) {  
        model.Health -= amount;  
        view.Render(model.Health);  
    }  
}
```

UI Software - MVC

Advantages

Allows for a separation of concerns. By separating the presentation from the data, it makes it easier to change one of them without affecting the other.

Presentation objects more reusable. Separating the user interface from the data allows UI components to be reused. It also means that a model can be reused with more than one view.

The separation of the presentation from the business logic and data allows developers to specialize in either frontend or backend development.

UI Software - MVC

Disadvantages

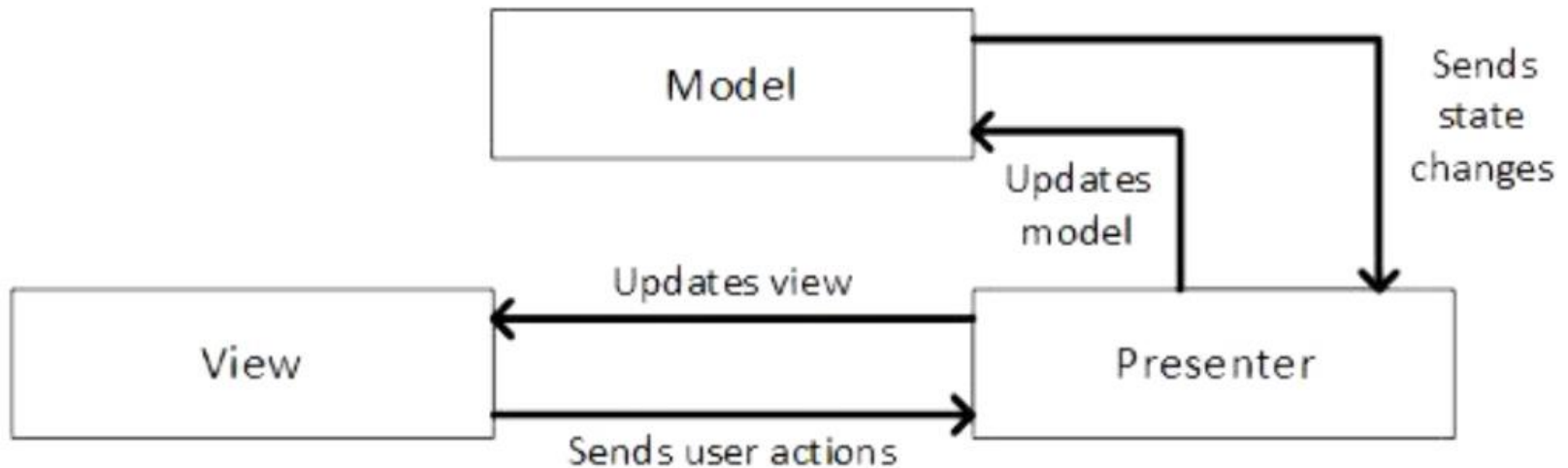
Difficult to achieve a complete separation. For example, adding a new field to the application will require a change in both the data and the presentation.

If your development team is not set up so that developers are focused on either frontend or backend development, this does require developers to be skilled in both areas (full-stack developers). This will require developers who are skilled in multiple technologies.

If a model is very active and is providing notifications directly to views, frequent changes to a model may result in excessive updates to views.

UI Software - MVP

Model-View-Presenter (MVP)



UI Software - MVP

Model: Similar as MVC pattern. It interacts with the database to retrieve and update data. The model receives messages from the presenter for updates and reports state changes back to the presenter. Models in the MVP pattern do not interact directly with views and only interact with the presenter.

View: The view is responsible for displaying the user interface and data. Each view in the MVP pattern implements an interface (view interface). As the user interacts with the view, the view will send messages to the presenter to act on the events and data. Presenters are loosely coupled with views through the view interface. Views are more passive in the MVP model and rely on the presenter to provide information on what to display.

UI Software - MVP

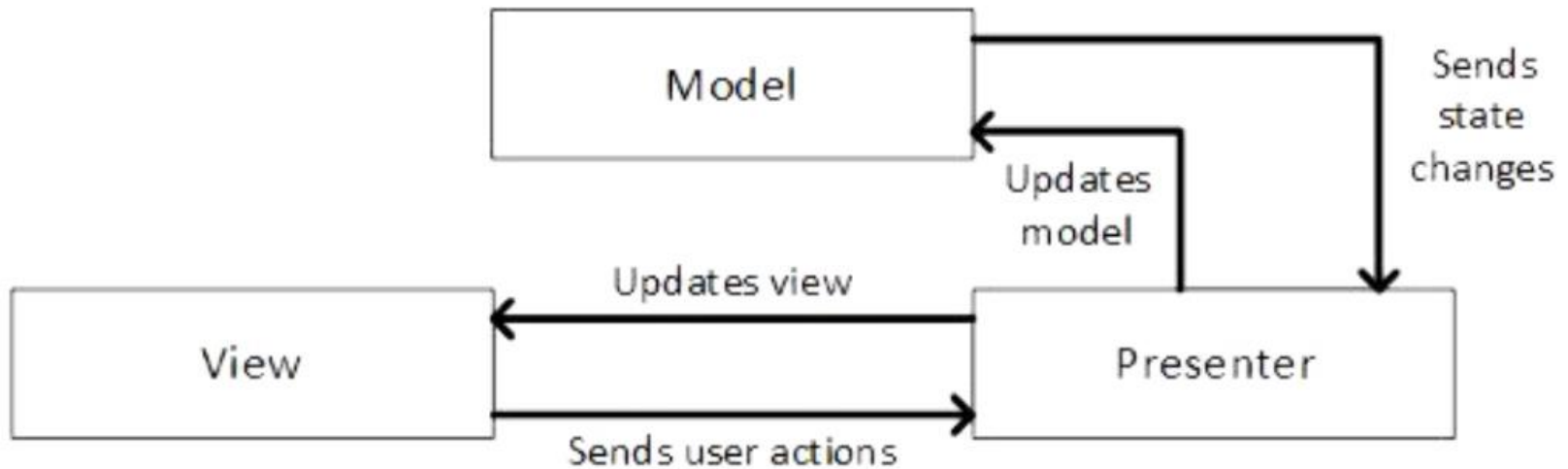
Presenter

The presenter is the intermediary between the model and the view. It interacts with both of them. Each view has a presenter and the view notifies the presenter of user actions. The presenter updates the model and receives state changes from the model.

A presenter will receive data from the model and format it for the view to display, taking an active role in presentation logic. Presenters encapsulate presentation logic and views play a more passive role.

Unlike the MVC pattern, where a controller can interact with multiple views, in the MVP pattern, each presenter typically handles one, and only one, view

UI Software - MVP



MVP – C# Example

```
// Model — pure data
class PlayerModel {
    public int Health { get; set; } = 100;
}

//View interface — presenter talks to this, not the concrete view
interface IPlayerView {
    void ShowHealth(int health);
    event Action OnHealClicked;
}

// Presenter — middleman, no Unity/UI dependencies
class PlayerPresenter {
    PlayerModel model;
    IPlayerView view;

    public PlayerPresenter(IPlayerView view) {
        this.model = new PlayerModel();
        this.view = view;
        view.OnHealClicked += () => {
            model.Health += 10;
            view.ShowHealth(model.Health);
        };
    }
}
```

UI Software - MVP

Advantage

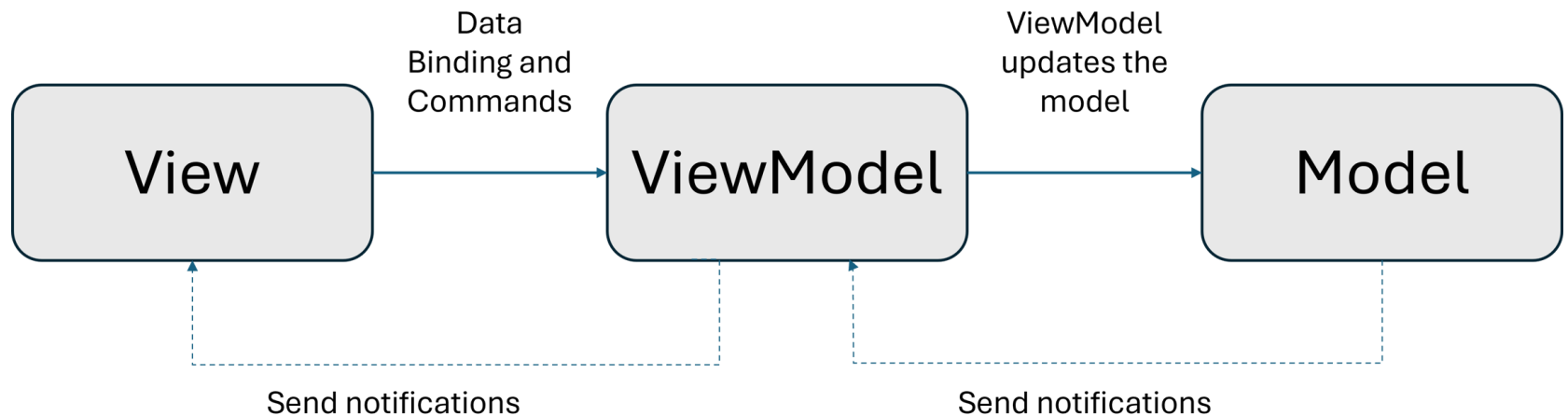
MVP offers precise control over interactions, excelling in scenarios that require dynamic user input or validation processes. It champions the separation of concerns and test-driven development, yielding resilient applications. MVP's Presenter mediates user interactions, enhancing code maintainability.

Disadvantage

The separation introduces complexity, potentially increasing code volume. The granular control of MVP necessitates a balance between benefits and intricacies.

UI Software - MVVM

Model-View-ViewModel (MVVM) Pattern



UI Software - MVVM

Model

Similar role as in MVC and MVP. It represents the business domain object and the data. The model uses the database to retrieve and update data. In MVVM applications, there may be direct binding with model properties. As a result, models commonly raise property changed notifications.

UI Software - MVVM

View

The view is responsible for the user interface. It is the part of the application that is visible to users. In the MVVM pattern, the view is active. Unlike a passive role where the view is completely manipulated by a controller or a presenter, and does not have knowledge of the model, in MVVM views are aware of the model and ViewModel.

While views handle their own events, they do not maintain state. They must relay user actions to the ViewModel, which can be done through a mechanism such as data binding or commands. A goal with the MVVM pattern is to minimize the amount of code in views.

UI Software - MVVM

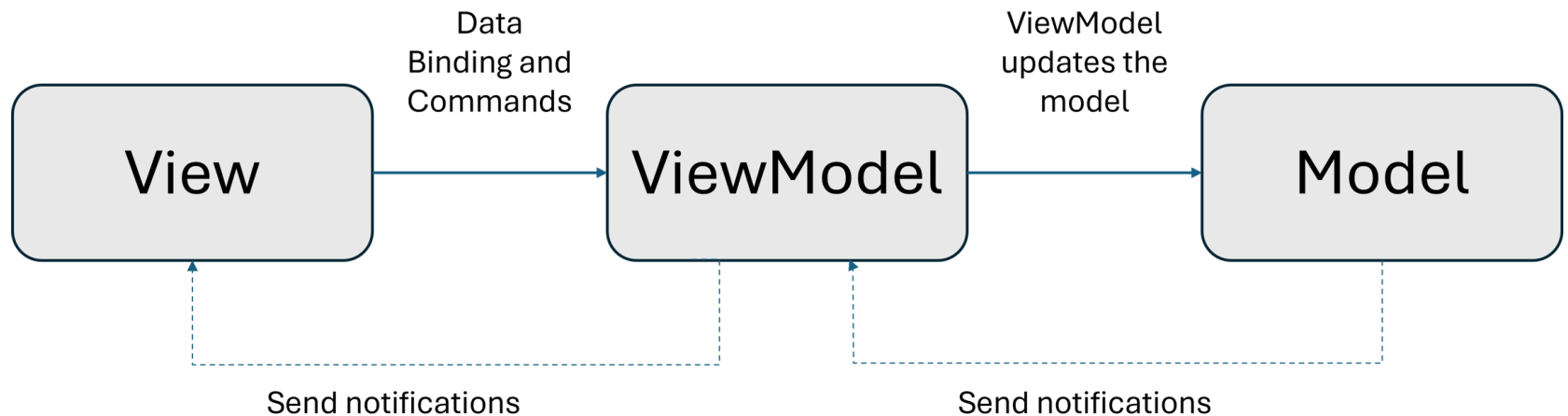
ViewModel

The ViewModel in the MVVM pattern is similar to the controller and presenter objects that we covered with the MVC and MVP patterns in that they coordinate between the view and the model.

ViewModels provide data to views for display and manipulation, and also contain interaction logic to communicate with views and models. ViewModels must be capable of handling user actions and data input sent from views. It is the ViewModel that contains navigation logic to handle moving to a different view.

Views and ViewModels communicate through multiple methods, such as data binding, commands, method calls, properties, and events.

UI Software - MVVM



MVVM – C# Example

// Model — pure data

```
class PlayerModel {  
    public int Health { get; set; } = 100;  
}
```

//ViewModel — exposes bindable properties, no UI code

```
class PlayerViewModel : INotifyPropertyChanged {  
    private PlayerModel model = new PlayerModel();  
    public event PropertyChangedEventHandler PropertyChanged;  
  
    private int health;  
    public int Health {  
        get => health;  
        set {  
            health = value;  
            PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(nameof(Health)));  
        }  
    }  
  
    public ICommand HealCommand => new RelayCommand(() => Health += 10);  
}
```

//View — binds to ViewModel properties automatically (WPF/Unity UI Toolkit)

```
// <TextBlock Text="{Binding Health}" />
```

Key difference: the **View binds directly** to ViewModel properties — no manual `view.ShowHealth()` calls needed.

UI Software - MVVM

Advantage

Its separation of concerns empowers designers and developers to collaborate while maintaining code integrity. Ideal for UI-intensive projects, MVVM introduces the ViewModel layer as a bridge between Model and View, enabling dynamic updates and interactions.

Disadvantage

Newcomers may encounter a learning curve due to the abstraction, and simple applications might find this elegance overly elaborate.

MVC vs MVP vs MVVM

Aspect	MVC	MVP	MVVM
Separation of Concerns	Basic	Better	Best
Data Flow	Two-way	One-way	One-way with data binding
View-Logic Relationship	Many-to-one	One-to-one	Many-to-one
Testability	Hard	Good	Best
Maintenance	Hard	Easy	Easy
Learning Curve	Easy	Easy	Harder
Performance	Can be slower due to tight coupling	Better performance with looser coupling	Smooth performance, especially for complex UIs
UI Updates	Controller updates View	Presenter updates View	ViewModel updates View through data binding
Dependency on UI Framework	High	Low	Low or no dependency
Scalability	Suitable for small-scale projects	Good for simple and complex projects	Ideal for large, data-heavy apps

MVC vs MVP vs MVVM

Pattern	Best for	Testability
MVC	Web (ASP.NET), simple apps	Medium
MVP	WinForms, Unity UI	High
MVVM	WPF (Microsoft), Unity UI Toolkit, React	Highest

Some companies adopting these different architectures are:

1.MVC: StackOverflow, GoDaddy, Visual Studio website Dell

2.MVP: Google (for some Android apps)

3.MVVM: Apple (for some iOS apps using SwiftUI), Angular framework, Vue.js framework

Event-Driven Programming (EDP)

The patterns define **how to organize code**, Event-driven programming (EDP) defines **how components communicate** within that organization.

EDP enables decoupled components to communicate by producing, detecting, consuming, and reacting to events. An event-driven program's flow is determined by events such as user actions, system changes, sensor outputs, or messages from other programs. Instead of following a linear sequence of instructions, the program waits for and responds to these events.

Event-Driven Programming (EDP)

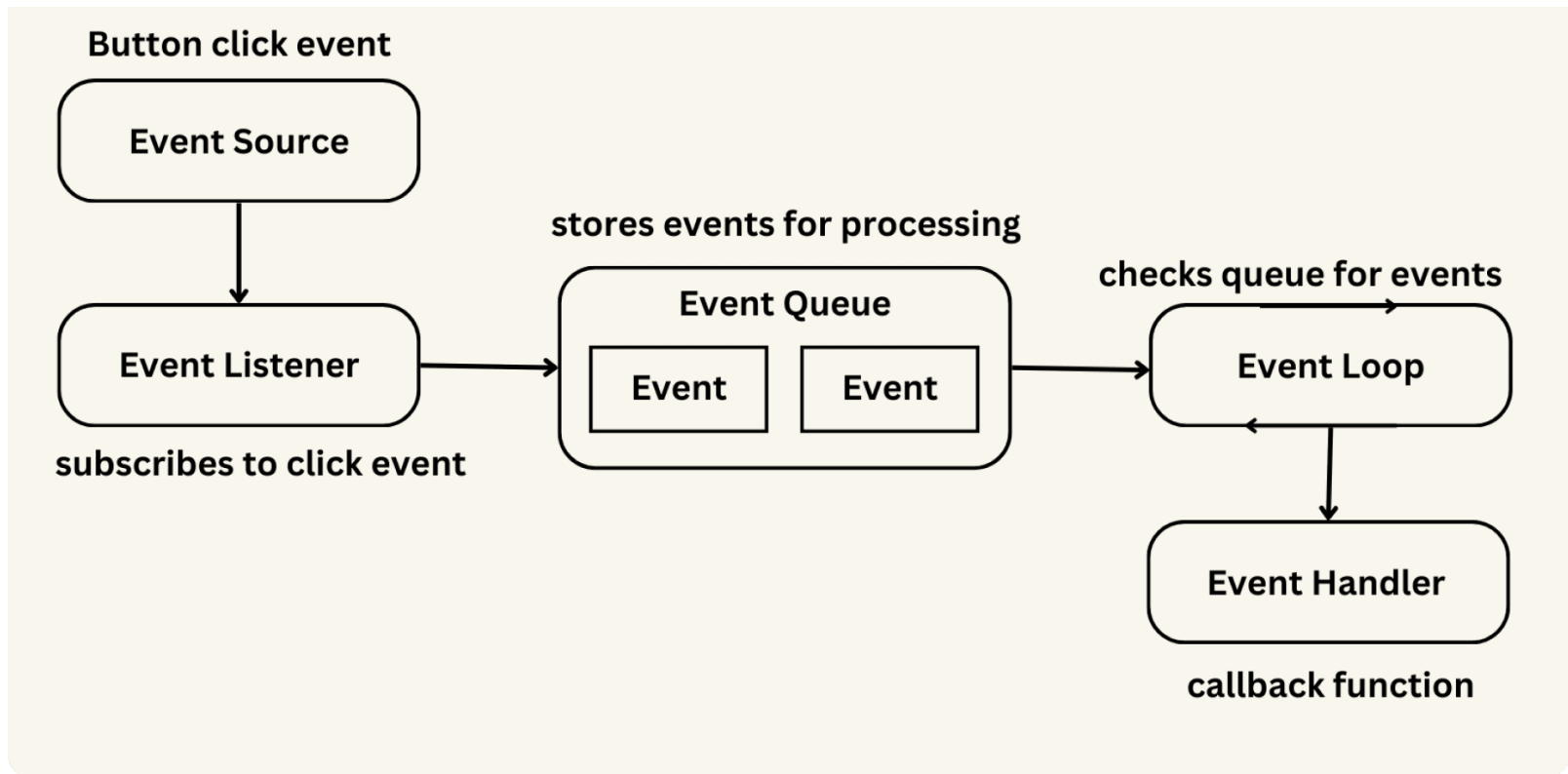
Events interact with three other key components:

Event sources (producers): generate and publish events such as button clicks or sensor data. They notify the application about these events without knowing which components are listening for or handling the events.

Event listeners (or subscribers): register for specific events and respond when they occur. When an event occurs, the listener is notified and calls the corresponding event handler, which executes the code to handle the event.

Event handlers: the functions or methods processing the event, perform tasks such as updating the user interface, making API calls, or triggering other events.

Event-Driven Programming (EDP)



Callback function

A **callback function** is a function passed into another function as an argument, which is then invoked inside the outer function to complete some kind of routine or action.

An Event Handler is a callback associated with an event (e.g., `addEventListener('click', fn)`). Every event handler is a callback, but not every callback is an event handler (e.g., `setTimeout(fn)`).

EDP - Example

```
//  
=====
```

// EVENT SOURCE — Unity's Button fires the event

```
//  
=====
```

```
class PlayerView : MonoBehaviour {  
    [SerializeField] Button damageButton; // Unity Button is the event source  
    [SerializeField] Button healButton;  
  
    // Expose buttons for the listener to subscribe to  
    public Button DamageButton => damageButton;  
    public Button HealButton  => healButton;  
}
```



```
// =====
// EVENT LISTENER — subscribes to button events
// =====
class PlayerController : MonoBehaviour {
    PlayerModel model = new PlayerModel();
    PlayerView view;

    void Start() {
        view = GetComponent<PlayerView>();

        // Subscribing = listening to button clicks
        view.DamageButton.onClick.AddListener(HandleDamagePressed); // register handler
        view.HealButton.onClick.AddListener(HandleHealPressed);    // register handler
    }

    void OnDestroy() {
        // Always unsubscribe to avoid memory leaks
        view.DamageButton.onClick.RemoveListener(HandleDamagePressed);
        view.HealButton.onClick.RemoveListener(HandleHealPressed);
    }
}
```

```
// =====
// EVENT HANDLERS — run when button is clicked
// =====

// Handles damage button click
void HandleDamagePressed() {
    model.Health -= 10;
    view.ShowHealth(model.Health);
}

// Handles heal button click
void HandleHealPressed() {
    model.Health += 10;
    view.ShowHealth(model.Health);
}
}
```

Polling

Polling is a method in which a device continuously checks the status of another device/component to determine whether it needs attention. This involves sending requests or queries to the device/component, waiting for a response, and repeating this process in a loop until the desired condition or data is obtained.

For example, a microcontroller can poll a sensor to read its value and determine whether data is available.

Simpler but consume more CPU

Polling - Example

```
class TemperatureSensor : MonoBehaviour {  
    [SerializeField] float pollInterval = 1f; // check every 1 second  
    float lastPollTime;  
    float temperature;  
  
    void Update() {  
        // Throttled polling — not every frame, but still polling  
        if (Time.time - lastPollTime >= pollInterval) {  
            lastPollTime = Time.time;  
            temperature = ReadSensor();  
            Debug.Log($"Temperature: {temperature}°C");  
        }  
  
        if (Keyboard.current.spaceKey.isPressed)  
            Debug.Log("Space HELD"); // fires every frame while held  
    }  
}
```

Polling vs Events

	Polling	Events
CPU usage	Always running	Only on change
State tracking	Manual	Automatic
Coupling	Checker knows the source	Source doesn't know listeners
Best for	Continuous input, smooth values	Discrete changes, one-time triggers
Worst for	Rare discrete events	Rapid continuous values

- If it happens **continuously** → poll
- If it happens **occasionally** → event